Ghost Seal — team rollout

Where it is stored, how each group uses it, and what makes it stick — v1.0, 2026-08-05

The one decision that matters

Do not roll this out as a tool people are asked to remember. Roll it out as a step in the publishing path that runs whether anyone remembers or not.

Every disclosure regime fails the same way: the mark is applied by whoever was diligent that afternoon. Six months later nobody can say which assets were marked, and that gap is the entire risk — an unsealed asset in market is worse than no policy, because it proves the policy exists and was not followed.

So: **seal on ingest, gate on publish, and let humans use the tool only for exceptions.**

Where it lives

1. A git repo — the source of truth

```
ghost-seal/
  seal_master.png        # the pinned seal geometry (85 KB, commit it directly)
  seal_master.json       # pinned mask hash + geometry table + font hashes
  ghost_seal.py          # apply / batch / verify / selftest
  build_master.py        # regenerates the master. Run only to change the standard
  verify_tree.py         # publish gate: fails if any image is unsealed
  tests/reference.png    # the fixture the CI determinism check runs on
  doc/                   # the spec document and this rollout note
  Dockerfile             # the pinned runtime
  .github/workflows/ci.yml
  CODEOWNERS
```

Protect `main`. Put this in `CODEOWNERS` so the seal itself cannot change quietly:

```
seal_master.png     @brand-lead @legal-counsel
seal_master.json    @brand-lead @legal-counsel
ghost_seal.py       @platform-owner
doc/                @brand-lead @legal-counsel
```

The mask hash is pinned in four places on purpose — `seal_master.json`, the spec document, the CI workflow, and the agent skill's metadata. Changing the seal means changing all four in one reviewed diff. That redundancy is the governance: there is no way to alter the house mark without it showing up as an approval request.

2. A container — the pinned runtime

The seal is scaled from the master with LANCZOS resampling. Resampling output can differ subtly between library versions, so "identical everywhere" needs a pinned interpreter, not just a pinned mask. Ship one image and have every lane use it — CI, the ingest worker, the web tool and any laptop.

```
FROM python:3.11.2-slim
RUN apt-get update && apt-get install -y --no-install-recommends \
      libimage-exiftool-perl && rm -rf /var/lib/apt/lists/*
RUN pip install --no-cache-dir "pillow==12.3.0" "numpy==2.4.6"
WORKDIR /app
COPY ghost_seal.py build_master.py verify_tree.py seal_master.png seal_master.json ./
RUN python3 ghost_seal.py selftest        # the build FAILS if the seal misbehaves
ENTRYPOINT ["python3", "ghost_seal.py"]
```

Tag it `ghost-seal:1.0.0` — same version number as the seal standard, never `latest`. The selftest inside the build means a broken image can never be published.

3. The asset store — sealed masters only

One bucket or DAM collection holds the **sealed masters**. Every crop, resize, thumbnail and platform variant is generated from those. Nothing downstream ever gets stamped: a shrunken seal is neither legible nor re-findable, and re-stamping a derivative is how you end up with two seals on one asset.

If your generation output currently lands in the same place designers pull from, split it: `incoming/` (raw model output) and `masters/` (sealed, publishable). The ingest worker moves assets between them. Publishing from `incoming/` becomes structurally impossible rather than discouraged.

4. The manifests — the evidence

`manifest.jsonl` lines are the per-asset audit record: source and output SHA-256, mask hash, geometry, relief mode, applied ΔL*, and UTC timestamp. Ship them to whatever you already query — a log store, a warehouse table, even an append-only bucket. The question you must be able to answer in minutes is "was this specific published asset sealed, when, and with which seal version". Without the manifests you can only answer it by re-examining pixels.

How each group uses it

Group	Lane	What they do
Nobody (automated)	**Ingest hook**	A worker watches the generation output location and seals every new asset into `masters/`. This is where 95% of volume should flow
Designers, marketers, social	**Internal web tool**	Drag images in, get sealed files plus a manifest back. Same container as CI, so output is identical
Engineers, ops	**Container CLI**	`docker run --rm -v $PWD:/w ghost-seal:1.0.0 batch /w/library --out /w/sealed --recursive --jobs 8`
CI / release	**Publish gate**	`verify_tree.py ./dist/assets --strict-metadata` fails the build if anything unsealed is about to ship
Agent workflows	**The skill**	`ghost-seal-disclosure` is installed per user account; teammates who work through the agent install it themselves. The repo and container remain the durable artifacts

The gate is the part people skip

`verify_tree.py` is what converts a policy into a fact. Wire it into the job that builds your site, exports a campaign package, or uploads to an ad platform. It walks a directory, checks each image for the visible seal (pixel correlation against the reference relief) and, with `--strict-metadata`, for the machine-readable provenance as well. Derivatives under 512px are reported as notes rather than failures, because they are expected to inherit the seal from their master.

Verified behaviour: a folder holding one sealed asset, one unsealed original and one 300px thumbnail exits 1 and names the unsealed file. The same folder with the unsealed file removed exits 0 under `--strict-metadata`.

Back catalogue

Run once, deliberately, as a migration rather than a background task:

1. Snapshot the current library (you want a rollback that is not "re-generate").
2. `batch --recursive` over it into a parallel sealed tree. Already-sealed and too-small files are skipped automatically, so the run is resumable and rerunnable.
3. Keep the manifest from that run forever — it is the record of when the standard was applied to historical work.

4. Swap the sealed tree in, then run `verify_tree.py --strict-metadata` over it as the acceptance test.

5. Re-derive live thumbnails and crops from the sealed masters. Assets already published on third-party platforms cannot be retro-sealed in place; decide with counsel whether they get replaced on next refresh or left as-is with the manifest showing the cut-over date.

Rollout sequence

Step	Owner	Done when
1. Repo created, hashes pinned, CODEOWNERS set	Platform	`main` protected, both CI jobs required
2. Container published to the internal registry	Platform	`ghost-seal:1.0.0` pullable, selftest green in build
3. Spec document circulated to brand + legal	Brand	Counsel has confirmed the metadata wording and scope
4. Ingest hook live on the generation output location	Platform	New assets appear in `masters/` already sealed
5. Publish gate added to the release job	Platform	An unsealed asset fails a build in staging
6. Web tool for non-technical users	Platform	Designers can seal an exception without asking anyone
7. Back catalogue migration	Brand + Platform	Manifest archived, `verify_tree` clean over the library

Steps 1, 2 and 5 are the minimum viable rollout. Everything else is convenience.

What to hand legal

- The spec document (particularly the regulatory framing and the known limits).
- The exact metadata fields written, so counsel can confirm the wording.
- A sample manifest line, so they can see what evidence exists per asset.
- The open questions: whether we act as provider or deployer per asset class, whether any of our imagery falls under the deep-fake disclosure duty, and whether we want C2PA signing on top of the unsigned IPTC tags.

Ask them for one thing in return: a signed-off decision on scope, recorded next to the seal version it applies to. When the standard changes, that pairing is what shows the change was deliberate.

Anti-patterns to refuse

• **A copy of the script in someone's local folder.** Determinism dies the moment two versions exist. Everything runs the container.

• **Stamping in the design tool by hand.** A hand-placed seal is a different seal.

• **Sealing derivatives.** Seal masters, derive everything else.

• **Two seal versions in circulation.** Worse than either alone, because it makes "which assets were marked" unanswerable.

• **Metadata without the visible seal, or the visible seal without metadata.** They answer to different audiences and ship together, always.

• **Treating the gate as advisory.** A gate that warns is a gate that is ignored.